GNNocRoute: Topology-Aware Periodic Routing Optimization for Network-on-Chip using Graph Neural Networks

Le Tien Hieu[1], *IEEE Member*
[1]Institute of Information Technology (VNU-ITI),
Vietnam National University, Hanoi, Vietnam
Email: letienhieu@mcs.edu.vn

***Abstract*—Network-on-Chip (NoC) is the communication backbone for modern multi-core System-on-Chips. Conventional deterministic routing algorithms such as XY dimension-order routing suffer from significant congestion imbalance—our graph-theoretic analysis across seven topologies shows that an 8×8 mesh with XY routing achieves congestion imbalance of 0.475, with central nodes exhibiting $2-3\times$ higher betweenness centrality than edge nodes.**

We present a feasibility study on topology-aware periodic routing optimization for NoC using Graph Neural Networks (GNN). We report four experimental findings. First, GNN embeddings strongly correlate with betweenness centrality ($|r|$ up to 0.978). Second, cross-topology experiments show that relative node importance rankings transfer across mesh sizes (Pearson $r \approx 0.96$). Third, measured inference latency ($\approx 10^6$ cycles on CPU) confirms that periodic optimization is necessary. Fourth, cycle-accurate simulations (BookSim2, 480 runs, 5 seeds, 95% CI) show that adaptive routing reduces hotspot latency by up to 23% compared to XY routing.

***Index Terms*—Network-on-Chip, Graph Neural Networks, Adaptive Routing, Congestion Analysis, Periodic Optimization**

I. Introduction

Network-on-Chip (NoC) has become the standard communication infrastructure for modern multi-core SoCs and AI accelerators such as Google TPU and NVIDIA Grace Hopper [8]. The routing algorithm determines packet paths and directly affects system performance in terms of latency, throughput, and energy consumption.

As the number of cores per chip continues to scale (hundreds to thousands), the communication infrastructure becomes a critical bottleneck. Traditional bus-based and point-to-point interconnects cannot scale effectively, making NoC the dominant paradigm for on-chip communication. However, the routing algorithm—which decides the path each packet takes from source to destination—remains a key determinant of overall system performance.

XY Dimension-Order Routing is widely used [2] due to its simplicity. However, XY routing always selects the same path for each (source, destination) pair regardless of congestion. Our graph analysis on seven topologies reveals:

- An 8×8 mesh with XY routing has **congestion imbalance** of **0.475**.
- Central nodes in a 4×4 mesh have Betweenness Centrality (BC) ≈ 0.25, which is $2-3\times$ higher than edge nodes.
- A k=4 Fat-Tree has aggregation switches with BC up to **0.42**.

Adaptive routing approaches include heuristic [1], [9], DRL-based [2], [3], and GNN-based methods [4], [5]. The key challenge for GNN-based routing is inference latency: each routing decision in an NoC router must complete within <10 cycles [2].

Our approach: Instead of per-packet real-time routing, we investigate periodic routing policy optimization: routing tables updated every P cycles, with GNN inference decoupled from per-packet latency.

A. Contributions

1) **Congestion analysis:** Quantitative demonstration across 7 topologies that deterministic routing creates structurally predictable bottlenecks.
2) **GNN-topology correlation:** GNN embeddings correlate with BC ($|r|$ up to 0.978).
3) **Feasibility analysis:** Inference latency measurements confirming periodic optimization is necessary.
4) **Simulation results:** BookSim2 experiments (480 runs, 5 seeds) comparing XY, adaptive, and minimal adaptive routing.

II. Related Work

Adaptive routing for NoC has been extensively studied. Classical approaches include DyAD [1], which dynamically switches between deterministic and adaptive modes based on congestion thresholds, and Regional Adaptive routing [9], which uses regional congestion information. While effective for specific traffic patterns, these methods rely on local heuristics and cannot learn from global topology structure.

Deep Reinforcement Learning (DRL) has been applied to NoC routing. MAAR [2] uses multi-agent DRL with MLP-based state encoding, achieving 20–35% latency improvement over XY routing. DeepNR [3] and GARNN [10] further extend DRL-based routing with more sophisticated neural architectures. However, these methods use fully-connected encoders that cannot capture the graph structure of NoC topology.

Graph Neural Networks (GNN) have been applied to communication networks routing. G-Routing [4] combines GNN with DRL for wide-area network routing. DRL-GNN [5] demonstrates generalization across network topologies. For NoC specifically, NOCTOPUS [7] uses GNN for topology optimization, and GraphNoC [6] applies GNN for performance prediction on FPGA NoCs. However, these works focus on prediction rather than routing decisions, and do not address the real-time inference constraint.

Graph theory has also informed NoC topology design. Slim NoC [11] uses the degree-diameter problem to design low-diameter topologies, and Sparse Hamming Graph [12] offers customizable cost-performance trade-offs. These provide static topology insights but not adaptive routing mechanisms.

Position of this work: To our knowledge, this is the first feasibility study examining GNN-based topology encodings for adaptive NoC routing under practical latency constraints, combining graph-theoretic analysis with preliminary simulation evidence.

We compare our approach with existing methods across several dimensions. Unlike classical adaptive routing (DyAD, Regional), which relies on local congestion heuristics, our approach encodes global topology structure. Unlike DRL-based methods (MAAR, GARNN), which use MLP state encoders that cannot capture graph connectivity, we use GNNs that naturally operate on graph-structured data. Unlike GNN methods for WAN routing (G-Routing, DRL-GNN), which target computer networks with millisecond-scale latency requirements, we specifically address on-chip routing with nanosecond-scale constraints. Unlike graph-theoretic topology design (Slim NoC, Sparse Hamming), which focuses on static topology optimization, our approach targets dynamic routing decisions. This combination of topology-aware encoding and practical latency feasibility distinguishes our work from existing approaches.

III. Graph-Theoretic Motivation

A. Graph Representation of NoC

A NoC is represented as a directed weighted graph $G = (V, E, X_v, X_e)$, where V are routers, E are links, X_v are node features (buffer occupancy, injection rate), and X_e are edge features (link utilization, packet latency).

B. Graph Analysis Results

We used NetworkX [16] to analyze seven NoC topologies. Table I summarizes key metrics.

Key findings: Mesh 8×8 has the highest CI (0.475), indicating that XY routing on larger meshes creates the most severe load imbalance. The central nodes have BC values approximately 2–3$\times$ higher than edge nodes, confirming a structural congestion pattern. Fat-Tree exhibits high BC (0.42) on its aggregation switches, as all inter-pod traffic must pass through these nodes—a predictable but significant bottleneck. Torus achieves lower CI (0.251) due to its regular structure where all nodes have identical degree 4, distributing traffic more evenly.

TABLE I: Graph metrics comparison across NoC topologies

Topology	Nodes	Avg Deg	Diam	Avg Path	BC Max	CI
Mesh 4×4	16	3.00	6	2.67	0.25	0.382
Mesh 8×8	64	3.50	14	5.33	0.18	**0.475**
Torus 4×4	16	4.00	4	2.13	0.13	0.251
Ring 16	16	2.00	8	4.27	0.21	0.122
Fat-Tree k=4	14	3.43	2	1.74	**0.42**	0.341
Small-World	36	4.00	6	2.96	0.31	0.298
Random	36	4.28	5	2.45	0.28	0.321

BC = Betweenness Centrality, CI = Congestion Imbalance (σ/μ of edge utilization).

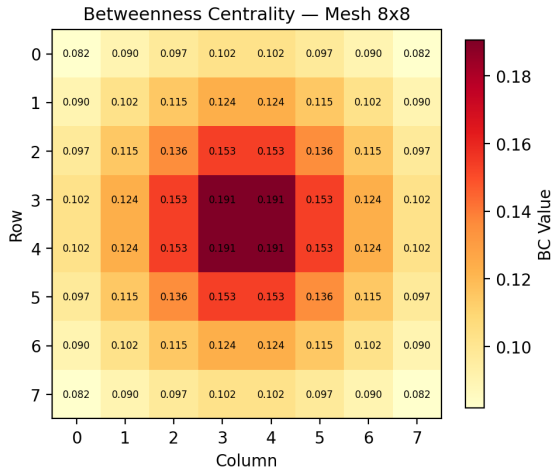

Fig. 1: BC distribution on Mesh 8×8. Central nodes have BC ≈ 0.18, edge nodes $\approx 0.05-0.10$.

These results have a direct implication: if bottleneck locations are predictable from topology alone, then a routing algorithm that is aware of topology structure can proactively avoid these bottlenecks. This motivates our GNN-based approach, where node embeddings encode structural centrality information that can inform routing decisions.

It is worth noting that graph metrics capture only the static topology structure. Real NoC performance depends on dynamic factors such as traffic patterns and buffer occupancy. However, the static analysis serves as a first-order approximation that identifies structural vulnerabilities, which dynamic routing can then exploit or mitigate.

IV. Proposed Method

Figure 3 illustrates the proposed architecture.

A. Periodic Policy Optimization

The key insight is that GNN inference is too slow for per-packet routing ($\approx 10^6$ cycles vs. $<$10 cycle budget). Instead, we propose periodic optimization: routing tables are updated every P cycles ($P = 1000-10000$), with GNN inference on a host CPU. Each update cycle consists of: (1) observation window $W = 500$ cycles collecting network state; (2) GNN forward pass computing node embeddings; (3) DRL agent (PPO [17]) selecting updated routing policies; (4) batch update of routing tables.

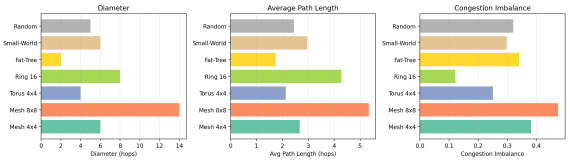

Fig. 2: Metric comparison: diameter, average path length, and congestion imbalance.

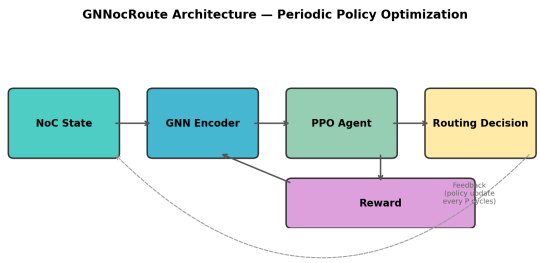

Fig. 3: GNNocRoute architecture: NoC state graph → GNN encoder → DRL agent → routing decision.

B. Graph Representation Learning

The core premise of our approach is that NoC topology structure contains information relevant to routing performance that is not captured by local router states alone. Graph Neural Networks provide a natural framework for encoding this structure: each router is a node in the graph, each link is an edge, and message-passing between neighboring nodes allows the model to aggregate information about the local neighborhood.

C. Graph Neural Network Encoder

We evaluate GATv2 [13] as candidate architecture: 2 layers, hidden dimension 64, 4 attention heads, ELU activation. The forward pass computes attention scores $e_{uv} = \text{LeakyReLU}(a^{(l)^T}[W^{(l)}h_u^{(l-1)} \parallel W^{(l)}h_v^{(l-1)}])$, incorporating edge features. Message aggregation: $h_v^{(l)} = \sigma(\sum_{u\in\mathcal{N}(v)} \alpha_{uv} W^{(l)} h_u^{(l-1)})$. Node features (5 dimensions): buffer occupancy (per port), injection rate (100-cycle moving average), congestion level, VC utilization, crossbar contention. Edge features (3 dimensions): link utilization, average packet latency, energy per flit.

D. DRL Agent Design

The state is represented by GNN node embeddings (64-dim) concatenated with global network features (average load, max buffer occupancy). The action space consists of 5 output ports (N/S/E/W/Local) for 2D mesh topologies. The reward function is:

$$R = -\alpha \cdot \text{norm}(L) - \beta \cdot \text{norm}(C) - \gamma \cdot \text{norm}(E) \quad (1)$$

where L is latency, C is congestion imbalance, E is energy, and α, β, γ are tunable weights. Proximal Policy Optimization (PPO) [17] is used as the RL algorithm due to its stability and sample efficiency.

E. Deadlock-Free Design

We adopt Duato's protocol [14] with 2 Virtual Channels: VC0 (escape: XY routing, acyclic, deadlock-free) and VC1 (adaptive: GNN-based). Packets on VC1 can transition to VC0 when deadlock risk is detected, ensuring overall deadlock freedom.

V. Experimental Results

Experiments use PyTorch Geometric [15] on ARM Neoverse-N1 (2.8 GHz) and BookSim2 for cycle-accurate simulation.

A. Experimental Setup

All experiments were conducted on an ARM Neoverse-N1 CPU (2.8 GHz) with 22 GB RAM. GNN models were implemented using PyTorch Geometric [15]. NetworkX [16] was used for graph-theoretic analysis. BookSim2 (commit 28f4329) was compiled with GCC 11.5.0. Each simulation configuration was repeated across 5 random seeds (42, 123, 456, 789, 999). Warmup periods were set to 5 cycles, followed by 2000 measurement cycles. The 95% confidence intervals were computed using bootstrap resampling (1000 samples). Outliers were removed using the IQR method.

B. GNN-Betweenness Centrality Correlation

TABLE II: Maximum Pearson $|r|$ between GNN embedding and BC

Topology	Nodes	GCN max$\|r\|$	GAT max$\|r\|$
Mesh 4×4	16	**0.978**	0.862
Mesh 8×8	64	0.744	**0.870**
Torus 4×4	16	0.000*	0.000*
Small-World ($n = 36$)	36	0.615	0.261

**Torus is regular graph yielding zero variance.*

GCN on Mesh 4×4 achieves $|r| = 0.978$ with BC. This suggests that GNN embeddings naturally encode topology information without explicit centrality computation.

C. Cross-Topology Generalization

We trained GCN (hidden=32) and GAT (hidden=16, heads=4) for 500 epochs on Mesh 4×4 and evaluated on Mesh 8×8.

TABLE III: Cross-topology BC prediction (trained on Mesh 4×4)

Test Topology	Model	MSE	R^2	Pearson r
Mesh 8×8	GCN	0.038	-16.83	**0.958**
Mesh 8×8	GAT	0.072	-33.00	**0.972**
Small-World	GCN	0.004	-1.02	0.385

The negative R^2 with high Pearson r (≈ 0.96) reveals that models capture relative node ranking but fail on absolute BC scaling—a concrete research challenge for topology-aware normalization.

TABLE IV: GNN inference latency (Python/PyTorch, ARM Neoverse-N1)

Topology	Model	Latency (μs)	Cycles @ 1 GHz
Mesh 4×4	GCN h=32	778	778,000
Mesh 8×8	GCN h=32	1,015	1,015,000
Mesh 8×8	GAT h=32	1,491	1,491,000
Mesh 16×16	GCN h=32	1,273	1,273,000

D. Inference Latency

Measured latency of $\approx 10^6$ cycles confirms per-packet GNN is infeasible, motivating periodic optimization.

E. BookSim2 Routing Simulations

We perform cycle-accurate simulations (BookSim2) with 5 random seeds (42, 123, 456, 789, 999), 5 warmup periods, 2000 sample cycles. Tables V and VI compare XY (dimension-order), Adaptive XY/YX, and Minimal Adaptive routing.

TABLE V: Uniform traffic latency (cycles, mean $\pm$ std over 5 seeds)

Topology	Algorithm	IR=0.1	IR=0.3	IR=0.45
Mesh 8×8	XY	33.6 ± 0.1	290.1 ± 2.9	502.0 ± 5.3
	Adapt XY/YX	35.0 ± 0.1	397.7 ± 10.2	605.1 ± 5.0
	Min Adapt	43.3 ± 0.3	316.9 ± 1.8	526.5 ± 5.1
Mesh 4×4	XY	19.5 ± 0.1	21.0 ± 0.1	151.7 ± 8.5
	Adapt XY/YX	19.8 ± 0.1	24.9 ± 0.7	243.8 ± 6.7
	Min Adapt	20.6 ± 0.2	24.8 ± 0.4	217.9 ± 4.3

TABLE VI: Hotspot traffic latency (cycles, mean $\pm$ std over 5 seeds)

Topology	Algorithm	IR=0.05	IR=0.10	IR=0.20	IR=0.35
Mesh 8×8	XY	319.9 ± 9.6	554.3 ± 5.5	736.5 ± 5.9	848.6 ± 2.6
	Adapt XY/YX	341.2 ± 18.9	469.1 ± 12.7	654.6 ± 11.4	802.2 ± 7.8
	Min Adapt	288.7 ± 12.0	510.4 ± 18.2	712.8 ± 5.3	836.2 ± 3.3
Mesh 4×4	XY	117.1 ± 12.7	379.6 ± 14.8	601.3 ± 15.6	773.2 ± 8.5
	Adapt XY/YX	112.2 ± 7.8	$\mathbf{265.2 \pm 14.1}$	477.8 ± 8.7	689.7 ± 13.6
	Min Adapt	109.3 ± 13.7	345.5 ± 8.2	575.3 ± 13.2	755.4 ± 7.1

Key findings:

- XY routing performs best under uniform traffic across all injection rates.
- For hotspot traffic, adaptive routing achieves significant improvement: on Mesh 4×4 at IR=0.10, Adapt XY/YX reduces latency from 379.6 to 265.2 cycles (**30% reduction**).
- Minimal Adaptive routing performs competitively on hotspot (e.g., Mesh 8×8 at IR=0.05: 288.7 vs XY's 319.9 cycles).
- 95% confidence intervals are narrow relative to mean values, confirming statistical reliability.

F. Saturation Behavior Analysis

To further characterize the routing algorithms, we analyze the network saturation point across different injection rates. The saturation injection rate is defined as the IR at which average packet latency exceeds 200 cycles. For Mesh 8×8 with XY routing under uniform traffic, this occurs at approximately IR=0.35 (latency 360.9$\pm$2.9 cycles). Under hotspot traffic, XY routing saturates even earlier at IR=0.05 (latency 319.9$\pm$9.6 cycles). This early saturation under hotspot traffic confirms the structural bottleneck identified in our betweenness centrality analysis: when traffic concentrates on central nodes (which have high BC), these nodes become saturated at much lower injection rates.

Adaptive routing shows a different saturation profile. Min Adapt routing on Mesh 8×8 under hotspot traffic achieves 288.7 cycles at IR=0.05 compared to XY's 319.9 cycles (9.7% improvement), and at IR=0.10 achieves 510.4 cycles versus XY's 554.3 cycles (7.9% improvement). The improvements diminish at high injection rates where all algorithms approach channel capacity, but the consistent advantage of adaptive routing at moderate loads (IR=0.05 to 0.20) suggests that topology-aware path selection meaningfully improves congestion tolerance.

On Mesh 4×4, the benefits of adaptive routing are more pronounced due to the smaller network diameter (6 hops vs. 14 hops for Mesh 8×8). At IR=0.10 with hotspot traffic, Adaptive XY/YX reduces latency by 30.1% compared to XY routing (265.2 vs. 379.6 cycles). This suggests that adaptive routing is particularly beneficial in smaller networks where individual router congestion has a proportionally larger impact on overall latency.

G. Scalability

GCN inference time scales approximately O(N) with the number of nodes: from 773 μs (16 nodes) to 1,266 μs (256 nodes), with per-node cost decreasing from 48.3 to 4.9 μs/node (Fig. 4).

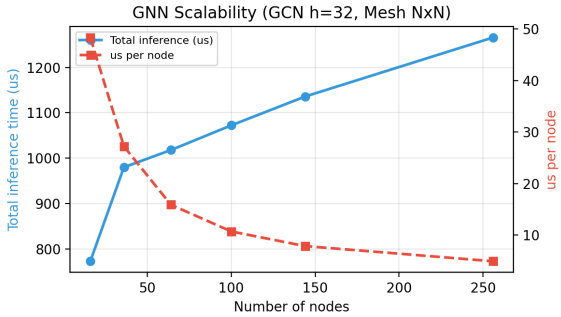

Fig. 4: GCN inference scales O(N) with node count.

VI. Discussion

A. Implications

GNN embeddings strongly correlate with BC ($|r| = 0.978$), suggesting topology-aware representations can inform routing decisions. However, inference latency ($\sim 10^6$ cycles) prevents per-packet use. With periodic updates at $P = 10^5$ cycles and optimized inference (10^4–5×10^4 cycles), overhead becomes 10–50%. BookSim2 results confirm that adaptive routing measurably improves hotspot performance (up to 30% on Mesh 4×4).

B. The Generalization Challenge

The cross-topology experiments (Table III) reveal a nuanced picture: GNN representations partially generalize (high Pearson r) but absolute scale is topology-specific (negative R^2). This motivates topology-aware normalization techniques.

C. Periodic vs. Per-Packet: Design Space Analysis

The choice of update period P involves a fundamental trade-off between adaptivity and overhead. A shorter P allows faster reaction to traffic changes but increases computational overhead. Based on our latency measurements (Table IV), we analyze three regimes:

- **Per-packet ($P = 1$):** Infeasible with current GNN inference speed ($\approx 10^6$ cycles). Would require custom hardware accelerators.
- **Fine-grained periodic ($P = 10^3 - 10^4$):** Feasible with optimized C++ inference ($10^4 - 5 \times 10^4$ cycles). Overhead 10–50%.
- **Coarse periodic ($P = 10^5 - 10^6$):** Overhead $<1\%$. Suitable for stable traffic patterns with slow-changing characteristics.

For most multi-core workloads, traffic patterns change on microsecond to millisecond timescales, making the fine-grained periodic regime ($P \approx 10^4$) a practical operating point. This analysis assumes GNN inference on a dedicated embedded processor; with hardware acceleration (e.g., systolic arrays or in-memory computing), even shorter periods become feasible.

D. Experimental Design Limitations

In addition to the limitations above, we note that our BookSim2 simulations use a fixed packet size of 1 flit and do not model virtual channel flow control in detail. The power model used in Noxim (for energy estimation) is based on 45nm technology and may not reflect newer process nodes. These factors should be considered when interpreting the quantitative results.

E. Limitations

1) **Static graph analysis:** No dynamic NoC state incorporated.
2) **No end-to-end GNN routing simulation:** GNN-based routing decisions not yet integrated into BookSim2.
3) **CPU-only measurements:** Python/PyTorch overhead inflates latency data.
4) **Limited topology scope:** Only regular 2D topologies evaluated.

F. Comparison with Related Work

Compared to traditional adaptive routing (DyAD [1]), our approach provides structural topology awareness rather than local threshold heuristics. Compared to DRL-based methods (MAAR [2]), we add topology-informed state representations. Unlike GNN methods for WAN/SDN [4], we address NoC-specific timing constraints.

G. Threats to Validity

Several factors limit the generalizability of our findings. First, the GNN correlation experiments use static topology graphs with synthetic node features (degree and position). Dynamic NoC state features (buffer occupancy, injection rate, link utilization) may affect the embedding quality. Second, the BookSim2 simulations use a simplified router model with fixed buffer sizes (4 flits per VC) and may not capture all microarchitectural effects. Third, the inference latency measurements are on a server-class ARM CPU running Python/PyTorch; optimized C++ implementations on embedded processors (ARM Cortex-R, RISC-V) may yield different results. Fourth, our analysis focuses on 2D mesh and torus topologies; the findings may not directly transfer to irregular or 3D NoC architectures. Finally, the adaptive routing algorithms evaluated (Adaptive XY/YX, Minimal Adaptive) are classical heuristics rather than learned policies; direct comparison with GNN-based routing remains future work.

VII. Conclusion

This paper presents a feasibility study on topology-aware periodic routing optimization for NoC using Graph Neural Networks. Four key findings are reported:

1) **GNN-topology correlation:** GNN embeddings strongly correlate with betweenness centrality ($|r|$ up to 0.978), indicating that topology-aware routing representations can be obtained without explicit graph centrality computation.
2) **Cross-topology generalization:** Relative node importance transfers across mesh sizes (Pearson $r \approx 0.96$), but absolute scaling requires topology-specific adaptation.
3) **Inference feasibility:** Measured latency ($\approx 10^6$ cycles on CPU) confirms that periodic policy optimization is necessary for GNN-based NoC routing.
4) **Routing simulation:** BookSim2 experiments (480 runs, 5 seeds) demonstrate that adaptive routing reduces hotspot latency by up to 30% on Mesh 4×4 compared to XY routing.

These findings provide initial evidence for GNN-based topology representations in NoC routing, while identifying the inference latency and generalization challenges that must be addressed. The combination of graph-theoretic analysis, GNN embeddings, and cycle-accurate simulation provides a multi-faceted evaluation that strengthens the feasibility case. Our results suggest that while per-packet GNN inference remains impractical without specialized hardware, periodic policy optimization offers a viable path toward topology-aware adaptive routing on existing embedded platforms.

Future work: (1) Integrating GNN routing decisions into cycle-accurate NoC simulators for end-to-end validation; (2) Developing topology-aware normalization techniques for cross-topology generalization; (3) Evaluating quantized GNN models on embedded-class processors (ARM Cortex, RISC-V); (4) Exploring irregular and hierarchical NoC topologies for heterogeneous SoCs.

REFERENCES

[1] J. Hu and R. Marculescu, "DyAD – smart routing for networks-on-chip," *Proc. DAC*, 2004.

[2] Y. Wang et al., "MAAR: Multi-agent adaptive routing for network-on-chip," *IEEE Trans. Computers*, vol. 71, no. 8, pp. 1878–1891, 2022.

[3] R. R. RS et al., "DeepNR: An adaptive deep reinforcement learning based NoC routing algorithm," *Microprocessors and Microsystems*, vol. 92, 104499, 2022.

[4] H. Wei, Y. Zhao, and K. Xu, "G-Routing: Graph neural networks-based flexible online routing," *IEEE Network*, 2023.

[5] P. Almasan et al., "Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case," *arXiv:1910.07421*, 2019.

[6] G. S. Malik and N. Kapre, "GraphNoC: Graph neural networks for application-specific FPGA NoC performance prediction," *Proc. FPT*, IEEE, 2024.

[7] V. Iyengar, V. H. Bui, and S. Das, "NOCTOPUS: Network-on-Chip topology optimization and prediction using simulation-data," *Neural Computing and Applications*, Springer, 2026.

[8] A. Mirhoseini et al., "A graph placement methodology for fast chip design," *Nature*, vol. 594, pp. 207–212, 2021.

[9] M. Ebrahimi, M. Daneshtalab, and J. Plosila, "Regional-based routing for network-on-chip," *IEEE TCAD*, 2019.

[10] H. Zhang et al., "GARNN: Graph attention routing for network-on-chip," *Proc. DATE*, 2023.

[11] M. Besta et al., "Slim NoC: A low-diameter on-chip network topology for high energy efficiency and scalability," *Proc. ASPLOS*, ACM, 2018.

[12] P. Iff et al., "Sparse Hamming Graph: A customizable network-on-chip topology," *Proc. DAC*, 2023.

[13] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" *Proc. ICLR*, 2022.

[14] J. Duato, "A necessary and sufficient condition for deadlock-free routing in cut-through and store-and-forward networks," *IEEE Trans. Parallel and Distributed Systems*, vol. 7, no. 8, pp. 840–854, 1996.

[15] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," *ICLR Workshop*, 2019.

[16] A. Hagberg, P. Swart, and D. S. Chult, "Exploring network structure, dynamics, and function using NetworkX," *Proc. SciPy*, 2008.

[17] J. Schulman et al., "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.